

Jihočeská univerzita v Českých Budějovicích
Přírodovědecká fakulta

Hlídací systém kotelny s využitím platformy Blynk

Semestrální práce předmětu IoT

Vít Horčíčka

České Budějovice 2026

Contents

Seznam zkratek	4
1 Úvod	5
1.1 Internet věcí (IoT)	5
1.2 Chytrá domácnost	5
1.3 Problematika spolehlivosti a řízení rizik	6
1.3.1 Specifika IoT a závislost na cloudu	6
2 Systém monitorování kotelny	8
2.1 Laskakit ESP32	8
2.2 Komunikace	8
2.3 Senzory	8
2.3.1 BME280	8
2.3.2 MQ135	8
2.4 Komponenty	9
2.4.1 Nepájivé pole (Breadboard)	9
2.4.2 LED diody	9
2.5 Software	9
2.5.1 Visual Studio Code	9
2.5.2 PlatformIO	9
2.5.3 Platforma Blynk	9
3 Návrh zapojení a signalizace	10
3.1 Signalizace LED diod	10
4 Návrh funkčnosti	10
4.1 Diagram programu	11
4.1.1 Odesílání dat do platformy Blynk	11
4.1.2 Logika nastavení hodnot prahů alarmu	12
4.2 Programový kód	13
4.2.1 Použité knihovny	13
4.2.2 Inicializace prostředí	14
4.2.3 Popis loopu	16
4.2.4 Popis hlídání podmínek	16
4.2.5 Komunikace s platformou Blynk	17
4.3 Platforma Blynk	19

4.3.1	Uživatelské rozhraní (Dashboard)	19
4.3.2	Notifikace a alarmy	20
4.3.3	Omezení bezplatné verze (Free Tier)	20
4.3.4	Alternativní řešení: Home Assistant	20
5	Závěr	21
5.1	Nalezené problémy	21
	Seznam použité literatury	23
A	Přílohy	24
A.1	Zdrojový kód ESP32	24

Seznam zkratek

- IoT - Internet of Things (Internet věcí)
- ESP32 - Expressif Systems 32 (Mikrokontrolér)
- I2C - Inter-Integrated Circuit (Komunikační protokol)
- MQTT - Message Queuing Telemetry Transport (Komunikační protokol)
- LED - Light Emitting Diode (Světlo emitující dioda)
- GND - Ground (Zem)
- 3V3 - 3.3 Volts (Napájecí napětí)
- VCC, VIN - Voltage Common Collector (Napájecí napětí)
- Wi-Fi - Wireless Fidelity (Bezdrátová technologie)
- API - Application Programming Interface (Rozhraní pro programování aplikací)

1 Úvod

Tato semestrální práce se zabývá návrhem a implementací hlídacího systému pro kotelnu s využitím platformy Blynk. V kontextu moderních chytrých domácností je automatizované monitorování a řízení klíčových systémů, jako je vytápění, nejen otázkou komfortu, ale především bezpečnosti a efektivity. Cílem této práce je vytvořit cenově dostupný a spolehlivý systém, který bude v reálném čase sledovat kritické parametry v kotelně – konkrétně teplotu, vlhkost, tlak a přítomnost nebezpečných plynů.

Práce je strukturována následovně: nejprve je představen koncept chytré domácnosti a problematika spojená s monitorováním kritických systémů. Následně je popsán hardware, včetně mikrokontroleru ESP32 a senzorů BME280 a MQ135, a softwarové řešení postavené na platformě Blynk. Další kapitoly se věnují detailnímu návrhu zapojení, signalizačním mechanismům a softwarové logice. V závěru jsou shrnuty dosažené výsledky, diskutovány nalezené problémy a navrženy možnosti dalšího rozšíření.

Výsledný systém umožní uživateli vzdáleně sledovat stav kotelny prostřednictvím mobilní aplikace, přijímat okamžitá varování v případě anomálií a mít tak neustálý přehled o bezpečnosti a funkčnosti vytápěcího systému.

1.1 Internet věcí (IoT)

Internet věcí (IoT) představuje rozsáhlou síť fyzických zařízení, vozidel, domácích spotřebičů a dalších předmětů, které jsou vybaveny senzory, softwarem a dalšími technologiemi umožňujícími připojení a výměnu dat s jinými zařízeními a systémy přes internet. Tato konektivita umožňuje vzdálený monitoring, sběr dat, analýzu a automatizaci procesů v reálném čase. Cílem IoT je zvýšit efektivitu, přesnost a ekonomický přínos v široké škále aplikací, od průmyslových systémů přes chytrá města až po osobní zařízení a domácnosti. V kontextu monitorovacích systémů, jako je ten v této práci, IoT poskytuje základní infrastrukturu pro sběr dat z fyzického světa a jejich zpřístupnění pro analýzu a akci. [4]

1.2 Chytrá domácnost

Chytrá domácnost, známá také jako inteligentní dům, je koncept bydlení, který zajišťuje optimální vnitřní prostředí pro život obyvatel. Toho je dosaženo souhrou stavební konstrukce, technologií, automatizovaného řízení a služeb. Cílem je zvýšit komfort, zlepšit zábavu, zajistit bezpečnost a snížit provozní náklady. Systém je obvykle modulární a jeho centrální jednotka automatizuje operace jako vytápění, osvětlení, zabezpečení a zábavu, přičemž vše je ovladatelné prostřednictvím různých rozhraní [3].

1.3 Problematika spolehlivosti a řízení rizik

Při návrhu jakéhokoli technického systému, jehož selhání může způsobit materiální škody nebo ohrozit zdraví, je nutné zohlednit principy řízení rizik. Cílem je identifikovat potenciální hrozby, analyzovat jejich pravděpodobnost a dopad a navrhnout opatření k jejich minimalizaci. Klíčovými pojmy jsou zde **spolehlivost** (schopnost systému plnit požadovanou funkci po danou dobu), **bezpečnost** (schopnost systému nezpůsobit nepřijatelné riziko) a **dostupnost** (pravděpodobnost, že systém bude v daném čase funkční). Ačkoliv monitorovací systém kotelny není formálně klasifikován jako kritický, aplikujeme tyto principy pro zajištění jeho robustnosti.

1.3.1 Specifika IoT a závislost na cloudu

U systémů spadajících do kategorie Internetu věcí (IoT) vstupuje do hry specifický rizikový faktor: závislost na externích službách a konektivitě. Využití cloudové platformy, jako je Blynk, přináší řadu výhod, ale také významná rizika, která je třeba řídit.

Výhody cloudového řešení

- **Vzdálený přístup:** Uživatel může kdykoliv a odkudkoliv kontrolovat stav kotelny prostřednictvím mobilní aplikace.
- **Okamžité notifikace:** V případě detekce anomálie (např. únik plynu) systém okamžitě odešle varování na mobilní zařízení uživatele, což umožňuje rychlou reakci.
- **Vizualizace a historie dat:** Cloudová platforma umožňuje přehledné zobrazení aktuálních i historických dat v grafech, což usnadňuje sledování trendů a diagnostiku problémů.
- **Jednoduchost implementace:** Hotová platforma výrazně zjednodušuje vývoj, protože není nutné programovat vlastní serverovou a mobilní aplikaci.

Nevýhody a rizika

- **Závislost na připojení k internetu:** Celá vzdálená funkčnost je závislá na stabilním internetovém připojení jak v místě instalace, tak na straně uživatele. Při výpadku se ztrácí možnost monitorování a přijímání notifikací.
- **Závislost na třetí straně:** Systém je závislý na dostupnosti a podpoře služby Blynk. Pokud má poskytovatel technický výpadek nebo službu v budoucnu ukončí, systém ztratí svou klíčovou funkcionalitu.

- **Potřeba lokální autonomie:** Z výše uvedených důvodů je nezbytné, aby zařízení mělo základní autonomní funkce. V této práci je to řešeno lokální optickou signalizací (LED diody), která varuje osoby v bezprostřední blízkosti zařízení i bez funkčního připojení k cloudu.
- **Bezpečnostní rizika:** Připojení k internetu otevírá systém potenciálním kybernetickým útokům. Je nutné dbát na silné zabezpečení sítě a přístupových údajů k platformě.

2 Systém monitorování kotelny

Pro monitorování prostředí v kotelně budou použity následující principy a technologie:

2.1 Laskakit ESP32

ESP32 je výkonný mikrokontrolér s integrovanou Wi-Fi a Bluetooth konektivitou, který je ideální pro IoT projekty. Jeho schopnost připojení k internetu umožňuje odesílání dat do cloudových služeb, jako je Blynk, a umožňuje vzdálené ovládání zařízení. ESP32 bude sloužit jako centrální jednotka pro sběr dat ze senzorů a komunikaci s platformou Blynk. [5]

2.2 Komunikace

Pro komunikaci mezi senzory a ESP32 bude využit protokol I2C, který umožňuje připojení více zařízení na stejnou sběrnici. Konkrétně je I2C komunikace realizována pomocí Hupšup konektoru od LaskaKit, což zjednodušuje zapojení. Data ze senzorů budou pravidelně odesílána do platformy Blynk, kde budou zpracovávána a vizualizována pro uživatele. Pro komunikaci s platformou Blynk bude využit protokol MQTT, který je optimalizován pro nízkou latenci a nízkou spotřebu energie, což je ideální pro IoT zařízení. ESP32 bude fungovat jako MQTT klient, který bude odesílat data do Blynk serveru a přijímat příkazy od uživatele. [1]

2.3 Senzory

2.3.1 BME280

Senzor BME280 je integrovaný senzor od společnosti Bosch Sensortec, který dokáže měřit atmosférický tlak, relativní vlhkost a teplotu. Vyznačuje se vysokou přesností a nízkou spotřebou energie, což jej činí ideálním pro monitorovací aplikace. Data ze senzoru BME280 jsou klíčová pro sledování podmínek v kotelně a identifikaci potenciálních abnormalit.

2.3.2 MQ135

Senzor MQ135 je senzor kvality ovzduší, který je schopen detekovat širokou škálu plynů, včetně amoniaku, sirovodíku, benzenu, alkoholu, kouře a oxidu uhličitého. Jeho citlivost na tyto plyny umožňuje monitorovat znečištění vzduchu a přítomnost nebezpečných látek v prostředí kotelny. Signál z MQ135 je důležitý pro posouzení celkové kvality ovzduší a včasné varování před rizikovými koncentracemi plynů. [6]

2.4 Komponenty

2.4.1 Nepájivé pole (Breadboard)

Pro snadné a rychlé prototypování bez nutnosti pájení je použit breadboard. Umožňuje flexibilní propojování jednotlivých komponent, jako jsou senzory a LED diody, s mikrokontrolérem ESP32.

2.4.2 LED diody

V projektu jsou použity tři LED diody různých barev (zelená, modrá, červená) pro vizuální signalizaci stavu systému. Jejich specifické funkce pro indikaci normálního provozu, varování a kritických stavů jsou popsány v kapitole 3.1.

2.5 Software

2.5.1 Visual Studio Code

Jako vývojové prostředí pro psaní kódu byl použit edit Visual Studio Code, který poskytuje širokou škálu rozšíření pro vývoj v jazyce C++ a práci s mikrokontroléry.

2.5.2 PlatformIO

PlatformIO je open-source ekosystém pro vývoj IoT projektů. Bylo použito jako rozšíření pro Visual Studio Code pro správu knihoven, kompilaci kódu a nahrávání firmwaru do mikrokontroléru ESP32.

2.5.3 Platforma Blynk

Pro vizualizaci dat a zasílání notifikací je využita platforma Blynk. Komunikace probíhá pomocí knihovny pr ESP32, která zjednodušuje připojení k serveru Blynk a výměnu dat.

3 Návrh zapojení a signalizace

Pro realizaci monitorovacího systému kotelny bude použito následující zapojení:

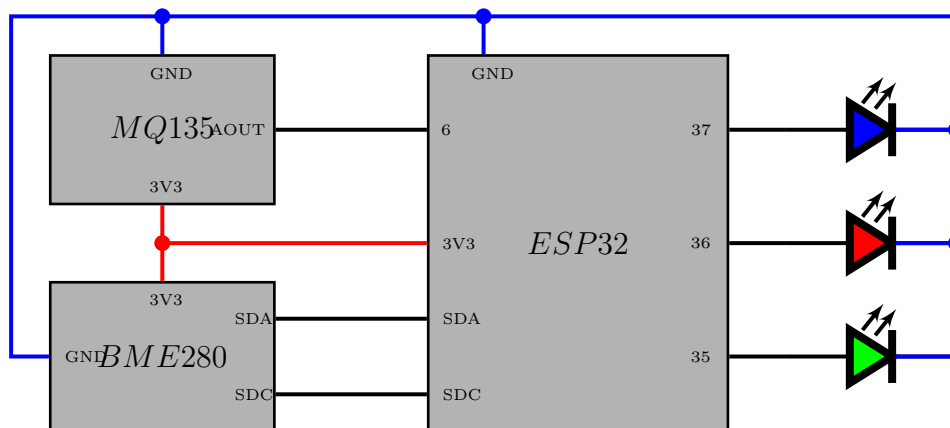


Figure 1: Diagram zapojení monitorovacího systému kotelny. Vytvořeno pomocí [2]

ESP32 bude připojen k senzoru BME280 pomocí I2C rozhraní a k senzoru MQ135 pomocí analogového vstupu. Napájení celého systému je zajištěno přes USB port mikrokontroléru ESP32, což umožňuje flexibilní a snadné napojení. Pro vizuální signalizaci různých stavů systému budou použity LED diody. Detailní rozvržení zapojení, včetně pinů a propojení jednotlivých komponent, je znázorněno v přiloženém diagramu.

3.1 Signalizace LED diod

Pro indikaci různých stavů systému budou použity LED diody s následujícími funkcemi:

- **Zelená LED:** Indikuje normální provoz a bezpečné podmínky v kotelně.
- **Modrá LED:** Indikuje běh celého programu, dalo by se říct že funguje jako "activity light".
- **Červená LED:** Indikuje kritické podmínky, jako je přehřátí, únik plynu nebo vysoká koncentrace škodlivých látek, které vyžadují okamžitou reakci.

4 Návrh funkčnosti

Systém bude v pravidelných intervalech sbírat data ze senzorů a kontrolovat je proti předem definovaným prahům. Pokud budou detekovány odchylky, systém bude aktualizovat stav LED diod a odesílat upozornění do platformy Blynk, kde bude uživatel informován o aktuálním

stavu kotelny a případných rizicích. V určitých časových intervalech bude systém také odesílat data do Blynk pro dlouhodobé sledování a analýzu trendů, což umožní uživateli lépe porozumět podmínkám v kotelně a případně optimalizovat její provoz.

4.1 Diagram programu

Pro lepší pochopení logiky programu pro monitorovací systém kotelny je níže uveden diagram, který znázorňuje hlavní funkční bloky a jejich vzájemné interakce.

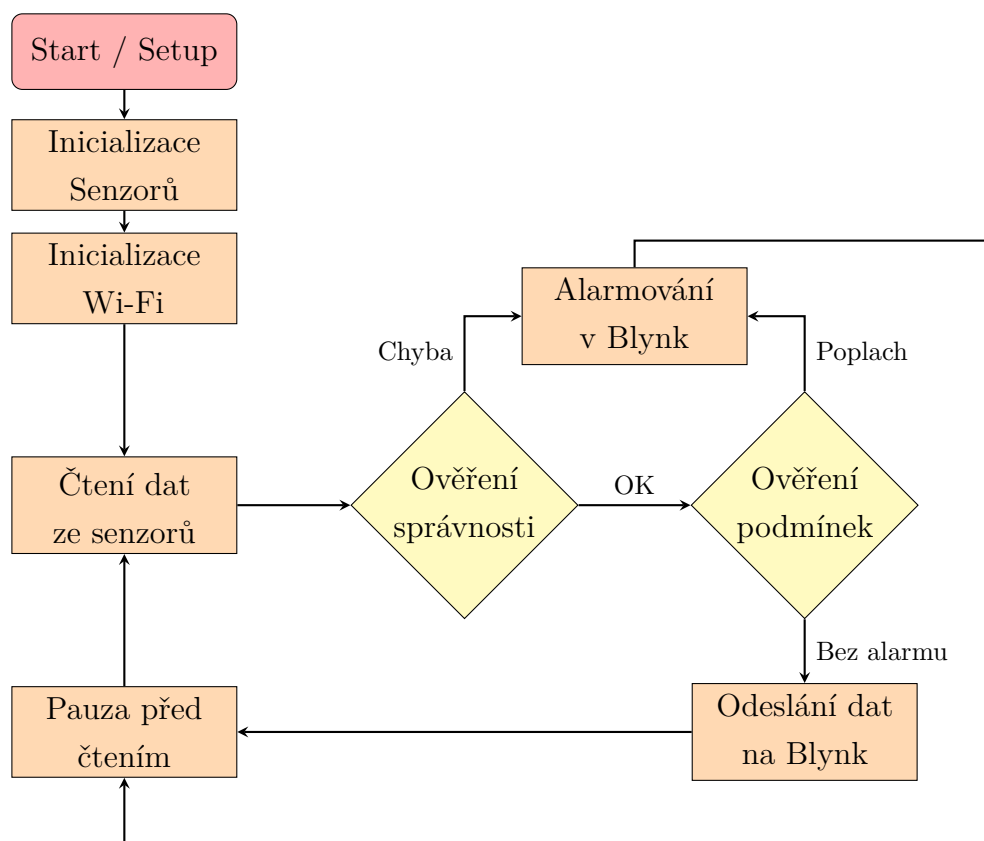


Figure 2: Diagram programu pro monitorovací systém kotelny.

4.1.1 Odesílání dat do platformy Blynk

Tento diagram popisuje jak se data kontrolují oproti pevně definovaných prahových hodnot změn. Pokud změna oproti naposledy odeslaným hodnotám bude větší tak je teprve odešle. To šetří odeslaná data pokud v místě nedošlo k žádné změně. Ale pokud by se data opravdu vůbec neměnily tak je to ošetřené aby to každých 15 minut odeslalo i bez žádné změny. To primárně slouží k udržení konzistence grafu.

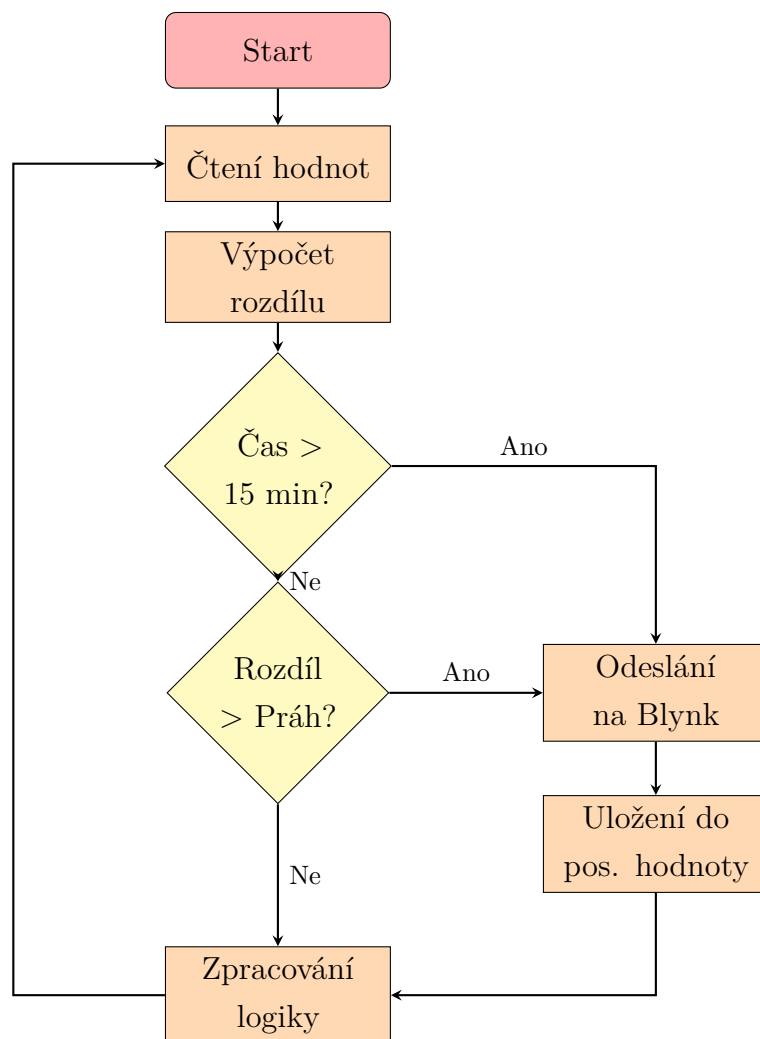


Figure 3: Diagram odesílání dat na cloud

4.1.2 Logika nastavení hodnot prahů alarmu

Program k možnosti alarmování musí mít nastavené prahové hodnoty a aby bylo omezeno nutné přehrání programu při změně tak se implementoval systém posílání nových nastavení a jejich následné uložení.

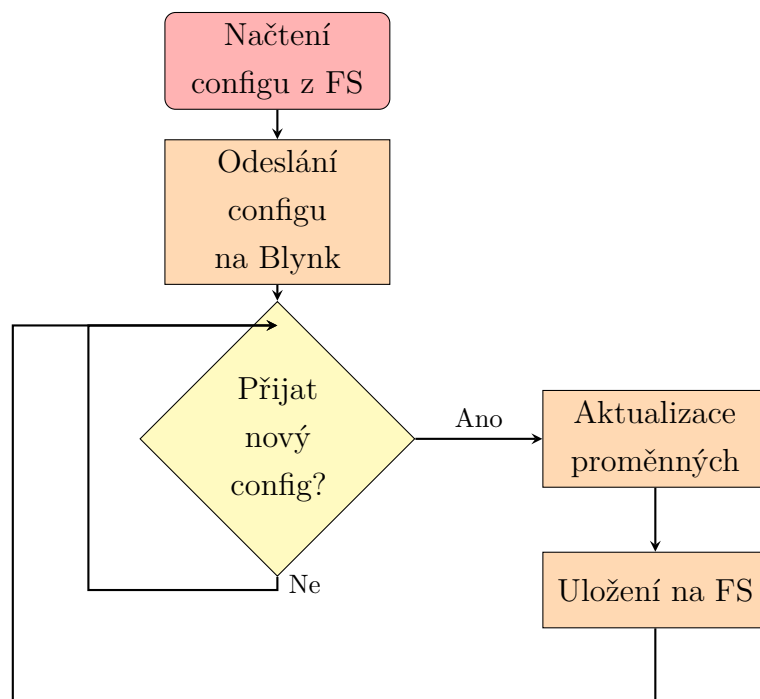


Figure 4: Diagram logiky nastavení alarmu

4.2 Programový kód

Programový kód pro ESP32 bude napsán v jazyce C++ a bude využívat knihovny pro komunikaci s Blynk, I2C a analogové vstupy. Kód bude strukturován tak, aby byl snadno čitelný a udržitelný, s jasně definovanými funkcemi pro sběr dat, kontrolu prahů, aktualizaci LED diod a komunikaci s platformou Blynk.

4.2.1 Použité knihovny

Pro správnou funkci zařízení je využíváno několik klíčových knihoven, které zajišťují komunikaci s hardwarem a cloudovými službami.

- `Wire.h`: Standardní knihovna pro I2C komunikaci, využívaná pro spojení se senzorem BME280.
- `Adafruit_BME280.h`: Knihovna od společnosti Adafruit pro snadné čtení hodnot teploty, vlhkosti a tlaku ze senzoru BME280.
- `MQ135.h`: Knihovna určená pro senzor kvality ovzduší MQ135, která zjednodušuje kalibraci a čtení hodnot znečištění.
- `WiFi.h`: Standardní knihovna pro ESP32, která umožňuje připojení k bezdrátové síti.

- **BlynkSimpleEsp32.h**: Oficiální knihovna platformy Blynk pro zjednodušení komunikace mezi mikrokontrolérem a Blynk serverem.
- **LittleFS.h**: Souborový systém pro ESP32, který slouží k ukládání konfiguračních souborů, v tomto případě prahových hodnot.
- **ArduinoJson.h**: Knihovna pro práci s datovým formátem JSON, využívaná pro serializaci a deserializaci nastavení ukládaných do LittleFS.

4.2.2 Inicializace prostředí

Inicializace systému probíhá ve funkci `setup()`, která je automaticky zavolána po startu mikrokontroléru. Tato funkce provádí sekvenci kroků nezbytných pro přípravu zařízení k provozu.

Načtení uloženého nastavení Na začátku se systém pokusí načíst dříve uložené prahové hodnoty z interního souborového systému LittleFS. Pokud soubor neexistuje, použijí se výchozí hodnoty.

```
1 void setup() {
2     Serial.begin(115200);
3     loadSettings();
4     // ...
```

Nastavení hardwarových periférií Dále dojde k inicializaci výstupních pinů pro LED diody, spuštění I2C sběrnice pro komunikaci se senzory a konfiguraci analogově-digitálního převodníku (ADC) pro senzor MQ135.

```
1 // ...
2 // LED indicators
3 pinMode(LED_OK_PIN, OUTPUT);
4 pinMode(LED_ERROR_PIN, OUTPUT);
5 pinMode(LED_Status_PIN, OUTPUT);
6
7 // I2C and ADC
8 Wire.begin(42, 2);
9 analogSetAttenuation(ADC_11db);
10 // ...
```

Inicializace senzorů Systém spustí komunikaci se senzorem BME280 a provede minutovou pauzu pro zahřátí a stabilizaci senzoru kvality ovzduší MQ135.

```
1 // ...
2 // BME280 sensor
3 bme.begin(0x77);
4
5 // MQ135 calibration in clean air
6 Serial.println("Heating MQ135 for 60 seconds...");
7 for(int i = 60; i > 0; i--) {
8     // ... LED blinking ...
9     delay(1000);
10 }
11 // ...
```

Připojení k síti a synchronizace Po inicializaci hardwaru se zařízení připojí k Wi-Fi a k serveru Blynk. Následně odešle své aktuální nastavení do mobilní aplikace, aby bylo zajištěno, že uživatelské rozhraní odpovídá stavu zařízení.

```
1 // ...
2 // Wi-Fi and Blynk connection
3 Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
4
5 // Send current settings to Blynk
6 sendSettingsToBlynk();
7 // ...
```

Nastavení časovačů a prvotní odeslání dat Nakonec se provede první měření, jehož výsledky se ihned odešlou do aplikace. Zároveň se nastaví dva časovače: jeden spouští kontrolu senzorů každou sekundu a druhý zajišťuje kompletní synchronizaci dat každých 15 minut.

```
1 // ...
2 // First reading and initialization
3 readSensors();
4 sendSensorDataBlynk(true);
5
6 // Timers for periodic tasks
7 timer.setInterval(1000L, oneSecondTask);
```

```

8   timer.setInterval(900000L, []() { sendSensorDataBlynk(true); });
9 }

```

4.2.3 Popis loopu

Hlavní smyčka programu, `loop()`, je záměrně minimalistická. Její jedinou starostí je pravidelně volat dvě funkce:

- `Blynk.run()`: Tato funkce zajišťuje veškerou komunikaci s platformou Blynk na pozadí. Zpracovává příchozí data (např. změny nastavení z aplikace) a odchozí data (hodnoty senzorů). Musí být volána co nejčastěji, aby byla zajištěna plynulá konektivita.
- `timer.run()`: Spravuje a spouští naplánované úlohy definované pomocí knihovny `BlynkTimer`. V našem případě se stará o periodické spouštění funkce `oneSecondTask` a patnáctiminutovou synchronizaci.

Díky tomuto přístupu je hlavní smyčka přehledná a veškerá logika je zapouzdřena v časovaných funkcích, což zjednodušuje údržbu kódu.

```

1 void loop() {
2     Blynk.run();
3     timer.run();
4 }

```

4.2.4 Popis hlídání podmínek

Funkce `checkThresholds()` je klíčovou součástí logiky pro hlídání bezpečnosti. Jejím úkolem je porovnat aktuálně naměřené hodnoty ze senzorů s prahovými hodnotami nastavenými uživatelem.

Nejprve se vyhodnotí celkový stav. Podmínka `currentOk` je pravdivá pouze tehdy, pokud jsou všechny čtyři měřené hodnoty (teplota, vlhkost, tlak a kvalita vzduchu) pod svými příslušnými prahy.

Následně funkce řeší logiku signalizace a alarmu.

- Pokud se stav změnil z *OK* na *ne-OK*, spustí se alarm. To zahrnuje nastavení příznaků pro rozsvícení červené LED, odeslání notifikace a události do platformy Blynk a výpis do sériové konzole. Příznak `alarmTriggered` zajistí, že se alarm spustí pouze jednou a nebude opakovaně posílat notifikace.

- Pokud se stav naopak vrátí z alarmujícího stavu zpět do normálu, alarm se zruší, zelená LED se rozsvítí a do Blynku se odešle zpráva o normalizaci stavu.

Tato logika zajišťuje, že uživatel je informován pouze o skutečných změnách stavu a není zahlcen zbytečnými zprávami.

```

1 // Kontrola prahových hodnot a alarmu
2 void checkThresholds() {
3     bool currentOk = temp < tempThreshold && hum < humThreshold &&
4         press < pressThreshold && airQuality < airQualityThreshold;
5
6     if (currentOk != ledOk || !alarmTriggered) {
7         ledOk = currentOk;
8         ledError = !ledOk;
9
10        if (!ledOk && !alarmTriggered) {
11            alarmTriggered = true;
12            Serial.println("\n[!!!] ALARM [!!!] Hodnoty překročily prah!");
13            Blynk.virtualWrite(V10, "ALARM");
14            Blynk.logEvent("alarm_event", "Alarm spusten!");
15        } else if (ledOk && alarmTriggered) {
16            alarmTriggered = false;
17            Serial.println("\n[OK] Hodnoty se vrátily pod prah. Alarm
18                zrušen.");
19            Blynk.virtualWrite(V10, "OK");
20            Blynk.logEvent("ok_event", "Hodnoty v normě");
21        }
22    }
23 }

```

4.2.5 Komunikace s platformou Blynk

Komunikace s platformou Blynk je stěžejním prvkem celého systému a je realizována pomocí knihovny BlynkSimpleEsp32.h. Tato knihovna abstrahuje složitost MQTT protokolu a umožňuje jednoduchou výměnu dat pomocí konceptu virtuálních pinů. Virtuální piny slouží jako most mezi hardwarem a widgety v mobilní aplikaci.

Odesílání dat ze zařízení Data jsou ze zařízení do aplikace posílána pomocí funkce Blynk.virtualWrite(pin, hodnota). Pro úsporu datového toku a snížení zátěže sítě se

data senzorů neposílají v každém cyklu, ale pouze pokud dojde k významné změně od poslední odeslané hodnoty, nebo pokud je vynuceno periodickým časovačem (každých 15 minut). Tuto logiku zajišťuje funkce `sendSensorDataBlynk()`.

```
1 // Odesilani hodnot senzoru do Blynk pouze pri vyznamne zmene
2 void sendSensorDataBlynk(bool force = false) {
3     bool sendTemp = force || fabs(temp - lastTemp) >=
4         TEMP_CHANGE_THRESHOLD;
5     // ... similar checks for other sensors ...
6
7     if (sendTemp) {
8         Blynk.virtualWrite(V0, temp);
9         lastTemp = temp;
10    }
11    // ...
12 }
```

Kromě telemetrických dat zařízení také posílá notifikace o alarmech pomocí funkce `Blynk.logEvent()`, která zaznamená událost do časové osy v aplikaci.

Příjem dat z aplikace Uživatelské rozhraní v aplikaci Blynk umožňuje měnit nastavení zařízení, jako jsou prahové hodnoty pro alarmy nebo zapnutí a vypnutí celého systému. Změna hodnoty widgetu v aplikaci vyvolá na straně ESP32 speciální funkci definovanou makrem `BLYNK_WRITE(pin)`. Pro každý ovladatelný virtuální pin ('V5' až 'V9') existuje jedna taková funkce, která přijme novou hodnotu, uloží ji do příslušné proměnné a následně zavolá `saveSettings()`, aby se změna trvale uložila do paměti zařízení.

```
1 // Prijem zmeny prahove hodnoty teploty z aplikace
2 BLYNK_WRITE(V5) {
3     tempThreshold = param.asFloat();
4     Serial.println("\n=====");
5     Serial.printf(">>> ZMENA PRAHU: TEPLOTA = %.2f C <<<\n",
6         tempThreshold);
7     Serial.println("=====\n");
8     saveSettings();
9 }
```

Udržování spojení a synchronizace Aby byla zajištěna neustálá a plynulá komunikace, je v hlavní smyčce `loop()` periodicky volána funkce `Blynk.run()`. Ta se na pozadí stará

o veškerou síťovou komunikaci, zpracování příchozích zpráv a odesílání dat ve frontě. Pro zajištění konzistence stavu mezi zařízením a aplikací se při startu mikrokontroléru volá funkce `sendSettingsToBlynk()`, která do aplikace odešle aktuálně platné nastavení.

4.3 Platforma Blynk

Blynk je komplexní IoT platforma, která umožňuje rychlý vývoj aplikací pro ovládání a monitorování hardwaru, jako je ESP32. Skládá se z cloudového serveru, mobilních aplikací pro iOS a Android a hardwarových knihoven. Její hlavní výhodou je jednoduchost, s jakou lze vytvořit plně funkční uživatelské rozhraní pomocí přetahování widgetů v mobilní aplikaci, bez nutnosti psát kód pro mobilní telefon. [[blynk__web](#)]

4.3.1 Uživatelské rozhraní (Dashboard)

Pro tento projekt bylo v aplikaci Blynk vytvořeno jednoduché uživatelské rozhraní (viz obrázek 5), které poskytuje veškeré potřebné informace a ovládací prvky na jedné obrazovce:

- **Zobrazení telemetrie:** Widgety pro zobrazení aktuální teploty, vlhkosti, tlaku a kvality ovzduší.
- **Stavové indikátory:** Textový indikátor, který jasně ukazuje, zda je systém v normálním stavu ("OK") nebo v alarmu.
- **Nastavení prahových hodnot:** Pro každou měřenou veličinu je k dispozici posuvník, kterým může uživatel pohodlně nastavit hraniční hodnotu pro spuštění alarmu.
- **Hlavní vypínač:** Přepínač, který umožňuje zapnout nebo vypnout celý monitorovací systém.

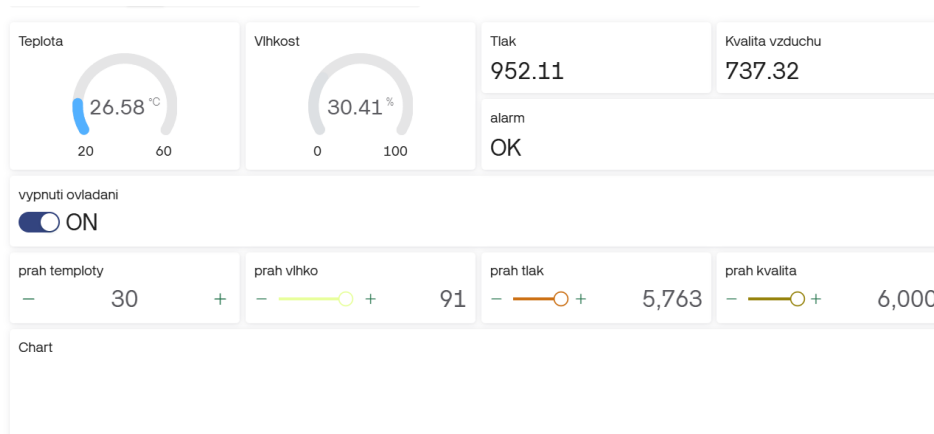


Figure 5: Náhled uživatelského rozhraní v aplikaci Blynk.

4.3.2 Notifikace a alarmy

Jednou z nejdůležitějších funkcí systému je schopnost aktivně upozornit uživatele na problém. V případě, že naměřené hodnoty překročí nastavené prahy, funkce `checkThresholds()` nejen aktivuje lokální signalizaci, ale také odešle do platformy Blynk kritickou událost.

```
1 // Fragment kodu z funkce checkThresholds()
2 if (!ledOk && !alarmTriggered) {
3     alarmTriggered = true;
4     Blynk.logEvent("alarm_event", "Alarm spusten!");
5     // ...
6 }
```

Příkaz `Blynk.logEvent()` vytvoří záznam v časové ose (Timeline) v mobilní aplikaci. Klíčové je, že v nastavení šablony (template) v platformě Blynk lze tuto událost ("alarm_event") nakonfigurovat tak, aby při svém výskytu automaticky odeslala **push notifikaci** na mobilní telefon uživatele. Díky tomu je majitel systému okamžitě informován o kritické situaci, i když aplikaci zrovna aktivně nepoužívá.

4.3.3 Omezení bezplatné verze (Free Tier)

Platforma Blynk je v základní verzi (Free Tier) nabízena zdarma, avšak s určitými omezeními. Pro vývoj a prototypování je tato verze dostatečná, ale pro produkční nasazení je třeba zvážit její limity. Mezi klíčové patří omezený počet zařízení, která lze k jednomu účtu připojit, a především limit na počet událostí (events), které lze za den odeslat. Událostí se rozumí například odeslání notifikace nebo změna hodnoty. Při překročení tohoto limitu může dojít k dočasnému zablokování komunikace. Právě z tohoto důvodu je v kódu implementována optimalizace, která odesílá data pouze při významné změně, a ne v pravidelných krátkých intervalech.

4.3.4 Alternativní řešení: Home Assistant

Jako alternativa ke komerčním cloudovým platformám, jako je Blynk, se nabízí open-source řešení. Nejpopulárnějším zástupcem je **Home Assistant**. Jedná se o výkonný systém pro domácí automatizaci, který běží lokálně (např. na Raspberry Pi) a dává uživateli plnou kontrolu nad jeho daty a zařízením.

- **Výhody:** Nezávislost na cloudu a připojení k internetu pro lokální funkce, obrovská komunitní podpora, tisíce integrací pro různá zařízení, žádné poplatky za provoz (kromě hardwaru).

- **Nevýhody:** Vyšší počáteční náročnost na instalaci a konfiguraci, nutnost spravovat vlastní server.

Pro tento projekt by integrace s Home Assistant byla logickým dalším krokem pro zvýšení spolehlivosti a rozšíření možností automatizace.

5 Závěr

Monitorovací systém kotelny s využitím platformy Blynk představuje efektivní řešení pro sledování a řízení podmínek v kotelně. Díky integraci senzorů BME280 a MQ135, spolu s výkonným mikrokontrolérem ESP32, je systém schopen poskytovat uživatelům důležité informace o stavu kotelny a umožnit jim rychle reagovat na potenciální hrozby. Ačkoliv tento systém není kritickým systémem v pravém slova smyslu, aplikuje některé z principů řízení rizik a bezpečnosti, které jsou klíčové pro zajištění bezpečného a spolehlivého provozu v prostředí kotelny.

5.1 Nalezené problémy

Během vývoje systému byly identifikovány následující problémy:

- **Omezená spolehlivost senzoru MQ135:** Tento senzor může být náchylný k falešným pozitivům a negativům, což může vést k nesprávným indikacím stavu vzduchu v kotelně.
- **Nedostatečná robustnost při výpadku Wi-Fi:** Současná implementace se spoléhá na výchozí chování knihovny Blynk, které je blokující. Při ztrátě Wi-Fi připojení se funkce `Blynk.run()` pokouší o znovupřipojení a po tuto dobu blokuje zbytek programu. V důsledku toho se v době výpadku nejen ztratí vzdálený dohled, ale zařízení přestane vykonávat i lokální operace, jako je čtení senzorů a aktualizace stavových LED diod.
- **Limity bezplatné verze platformy Blynk:** Jak již bylo zmíněno, bezplatná verze má přísný denní limit na počet zpráv (událostí). Do tohoto limitu se započítávají nejen data ze senzorů a alarmy, ale i interní "keep-alive" zprávy nutné pro udržení spojení. Ačkoliv je v kódu implementována optimalizace odesílání, při častých změnách hodnot hrozí vyčerpání limitu a dočasné znefunkčnění vzdáleného dohledu a notifikací.
- **Omezené možnosti rozšíření:** Systém je navržen pro konkrétní sadu senzorů a funkcí, což může omezit jeho schopnost přizpůsobit se budoucím potřebám nebo integraci s dalšími zařízeními v rámci chytré domácnosti.

- **Spoléhání na fungování senzorů:** Systém nepočítá se závadou senzorů a tím pádem v případě chybného senzoru nebude správně alarmovat a může způsobit pád celého systému.

Seznam použité literatury

- [1] *Blynk Documentation*. Blynk Inc. URL: <https://docs.blynk.io/>.
- [2] *Circuit to TikZ Designer*. FAU Erlangen-Nuremberg. URL: <https://www.circuit2tikz.tf.fau.de/designer/>.
- [3] *Intelligentní dům – Wikipedie*. Wikipedie, Otevřená encyklopedie. URL: https://cs.wikipedia.org/wiki/Intelligentn%C3%AD_d%C5%AFm.
- [4] *Internet věcí – Wikipedie*. Wikipedie, Otevřená encyklopedie. URL: https://cs.wikipedia.org/wiki/Internet_v%C4%9Bc%C3%AD.
- [5] *LaskaKit ESP32-S3 DevKit*. LaskaKit. URL: <https://www.laskakit.cz/laskakit-esp32-s3-devkit/?variantId=13145#relatedFiles>.
- [6] *Návod pro senzor plynu MQ-135*. dratek.cz. URL: <https://navody.dratek.cz/navody-k-produktum/senzor-plynu-mq-135.html>.

A Přílohy

A.1 Zdrojový kód ESP32

Listing 1: Hlavní program (main.cpp)

```
1 #include "secrets.h"
2 #include <Wire.h>
3 #include <Adafruit_I2CDevice.h>
4 #include <Adafruit_BME280.h>
5 #include <MQ135.h>
6 #include <WiFi.h>
7 #include <SPI.h>
8 #include <BlynkSimpleEsp32.h>
9 #include "settings_manager.h"
10
11 // -----
12 // Globalni promenne a nastaveni
13 // -----
14
15 // Mereni
16 float temp;
17 float lastTemp;
18 #define TEMP_THRESHOLD 0.5
19 float hum;
20 float lastHum;
21 #define HUM_THRESHOLD 1.0
22 float press;
23 float lastPress;
24 #define PRESS_THRESHOLD 1.0
25 float airQuality;
26 float lastAirQuality;
27 #define AIR_QUALITY_THRESHOLD 10.0
28
29 // Prahove hodnoty pro regulaci (lze menit pres Blynk)
30 float tempThreshold = 30.0;
31 float humThreshold = 60.0;
32 float pressThreshold = 1013.0;
33 float airQualityThreshold = 150.0;
```



```

34 bool onOff = false;
35 bool alarmTriggered = false;
36
37 // Prahove zmeny pro odesilani dat
38 #define TEMP_CHANGE_THRESHOLD 0.5
39 #define HUM_CHANGE_THRESHOLD 1.0
40 #define PRESS_CHANGE_THRESHOLD 1.0
41 #define AIR_QUALITY_CHANGE_THRESHOLD 10.0
42
43 // LED indikatory
44 bool ledOk = false;
45 bool ledError = false;
46 bool ledStatus = false;
47 #define LED_OK_PIN 35
48 #define LED_ERROR_PIN 36
49 #define LED_Status_PIN 37
50
51 // Senzory
52 MQ135 gasSensor = MQ135(6); // GPIO 6
53 Adafruit_BME280 bme;
54
55 // Buffer pro klouzavy prumer (pro MQ135)
56 const int MA_SAMPLES = 10;
57 float resBuffer[MA_SAMPLES];
58 float ppmBuffer[MA_SAMPLES];
59 int maIndex = 0;
60 bool maFilled = false;
61
62 // Blynk timer
63 BlynkTimer timer;
64
65
66 // -----
67 // Pomocne funkce
68 // -----
69
70 // Vypis hodnot senzoru do seriove linky
71 void printSensorData() {

```

```

72 Serial.println("---_Sensor_Readings_---");
73 Serial.printf("Temperature_: %.2f_C\n", temp);
74 Serial.printf("Humidity_: %.2f%%\n", hum);
75 Serial.printf("Pressure_: %.2f_hPa\n", press);
76 Serial.printf("Air_Quality_: %.2f_ppm\n", airQuality);
77 Serial.println("-----");
78 }
79
80
81 // Kontrola prahovych hodnot a alarmu
82 void checkThresholds() {
83     bool currentOk = temp < tempThreshold && hum < humThreshold &&
84         press < pressThreshold && airQuality < airQualityThreshold;
85
86     if (currentOk != ledOk || !alarmTriggered) {
87         ledOk = currentOk;
88         ledError = !ledOk;
89
90         if (!ledOk && !alarmTriggered) {
91             alarmTriggered = true;
92             Serial.println("\n[!!!_ALARM_!!!]_Hodnoty_prekrocily_prah!");
93             Blynk.virtualWrite(V10, "ALARM");
94             Blynk.logEvent("alarm_event", "Alarm_spusten!");
95         } else if (ledOk && alarmTriggered) {
96             alarmTriggered = false;
97             Serial.println("\n[OK]_Hodnoty_se_vratily_pod_prah._Alarm_
98                 zrusen.");
99             Blynk.virtualWrite(V10, "OK");
100             Blynk.logEvent("ok_event", "Hodnoty_v_norme");
101         }
102     }
103 }
104
105 // Cteni hodnot ze senzoru
106 void readSensors() {
107     temp = bme.readTemperature();
108     hum = bme.readHumidity();
109     press = bme.readPressure() / 100.0F;

```

```

108     airQuality = gasSensor.getCorrectedPPM(temp, hum);
109     printSensorData();
110 }
111
112 // Nastaveni LED indikatoru
113 void updateLEDs() {
114     digitalWrite(LED_OK_PIN, ledOk ? HIGH : LOW);
115     digitalWrite(LED_ERROR_PIN, ledError ? HIGH : LOW);
116     digitalWrite(LED_Status_PIN, ledStatus ? HIGH : LOW);
117 }
118
119
120
121 // Odesilani hodnot senzoru do Blynk pouze pri vyznamne zmene nebo
    vynucene
122 unsigned long lastForcedSend = 0;
123 const unsigned long FORCE_SEND_INTERVAL = 15UL * 60UL * 1000UL; //
    15 minut v ms
124
125 void sendSensorDataBlynk(bool force = false) {
126     bool sendTemp = force || fabs(temp - lastTemp) >=
        TEMP_CHANGE_THRESHOLD;
127     bool sendHum = force || fabs(hum - lastHum) >=
        HUM_CHANGE_THRESHOLD;
128     bool sendPress = force || fabs(press - lastPress) >=
        PRESS_CHANGE_THRESHOLD;
129     bool sendAirQuality = force || fabs(airQuality - lastAirQuality)
        >= AIR_QUALITY_CHANGE_THRESHOLD;
130
131     if (sendTemp) {
132         Blynk.virtualWrite(V0, temp);
133         lastTemp = temp;
134     }
135     if (sendHum) {
136         Blynk.virtualWrite(V1, hum);
137         lastHum = hum;
138     }
139     if (sendPress) {

```

```

140     Blynk.virtualWrite(V2, press);
141     lastPress = press;
142 }
143 if (sendAirQuality) {
144     Blynk.virtualWrite(V3, airQuality);
145     lastAirQuality = airQuality;
146 }
147 if (force) {
148     lastForcedSend = millis();
149 }
150 }
151
152
153
154 // Blynk: zmena prahovych hodnot a zapnuti/vypnuti regulace
155 BLYNK_WRITE(V5) {
156     tempThreshold = param.asFloat();
157     Serial.println("\n=====");
158     Serial.printf(">>>_ZMENA_PRAHU:_TEPLOTA_=%.2f_C_<<<\n",
159         tempThreshold);
159     Serial.println("=====\n");
160     saveSettings();
161 }
162
163 BLYNK_WRITE(V6) {
164     humThreshold = param.asFloat();
165     Serial.println("\n=====");
166     Serial.printf(">>>_ZMENA_PRAHU:_VLHKOST_=%.2f%%_<<<\n",
167         humThreshold);
167     Serial.println("=====\n");
168     saveSettings();
169 }
170
171 BLYNK_WRITE(V7) {
172     pressThreshold = param.asFloat();
173     Serial.println("\n=====");
174     Serial.printf(">>>_ZMENA_PRAHU:_TLAK_=%.2f_hPa_<<<\n",
175         pressThreshold);

```

```

175     Serial.println("=====\n");
176     saveSettings();
177 }
178
179 BLYNK_WRITE(V8) {
180     airQualityThreshold = param.asFloat();
181     Serial.println("\n=====");
182     Serial.printf(">>> ZMENA PRAHU: VZDUCH = %.2f ppm <<<\n",
183         airQualityThreshold);
184     Serial.println("=====\n");
185     saveSettings();
186 }
187
188 BLYNK_WRITE(V9) {
189     onOff = param.asInt() == 1;
190     Serial.println("\n!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
191     Serial.printf("!!! REGULACE JE NYNI %s!!!\n", onOff ? "ZAPNUTA (ON)" : "VYPNUTA (OFF)");
192     Serial.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!\n");
193     ledOk = onOff ? true : false;
194     updateLEDs();
195     saveSettings();
196 }
197
198 // Odeslani aktualnich nastaveni do Blynk
199 void sendSettingsToBlynk() {
200     Blynk.virtualWrite(V5, tempThreshold);
201     Blynk.virtualWrite(V6, humThreshold);
202     Blynk.virtualWrite(V7, pressThreshold);
203     Blynk.virtualWrite(V8, airQualityThreshold);
204     Blynk.virtualWrite(V9, onOff);
205 }
206
207 // Funkce pro periodickou kontrolu senzoru a aktualizaci stavu LED
208 // a odesilani dat do Blynk
209 void oneSecondTask() {
210     readSensors();
211     ledStatus = !ledStatus;

```

```

210
211     if (onOff) {
212         checkThresholds();
213     } else {
214         ledOk = false;
215         ledError = false;
216     }
217
218     updateLEDs();
219     sendSensorDataBlynk(false);
220 }
221
222
223 // -----
224 // Setup: inicializace systemu
225 // -----
226 void setup() {
227     Serial.begin(115200);
228     loadSettings();
229
230     // Inicializace pinu
231     pinMode(35, OUTPUT);
232     pinMode(4, OUTPUT);
233     digitalWrite(4, HIGH);
234     pinMode(47, OUTPUT);
235     digitalWrite(47, HIGH);
236     delay(1000);
237
238     // I2C a ADC
239     Wire.begin(42, 2);
240     analogSetAttenuation(ADC_11db);
241     analogReadResolution(10);
242
243     // LED indikatory
244     pinMode(LED_OK_PIN, OUTPUT);
245     pinMode(LED_ERROR_PIN, OUTPUT);
246     pinMode(LED_Status_PIN, OUTPUT);
247

```

```

248 // Senzor BME280
249 bme.begin(0x77);
250
251 // Kalibrace MQ135 v cistem vzduchu
252 Serial.println("Heating_MQ135_for_10_seconds...");
253 for(int i = 10; i > 0; i--) {
254     Serial.print("Heating...");
255     Serial.println(i);
256     ledOk = !ledOk;
257     ledError = !ledError;
258     ledStatus = !ledStatus;
259     updateLEDs();
260     delay(1000);
261 }
262 Serial.println("Everything_is_ready!");
263 Serial.println("-----");
264 ledOk = false;
265 ledError = false;
266 ledStatus = false;
267 updateLEDs();
268
269 // Pripojeni k Wi-Fi a Blynk
270 WiFi.mode(WIFI_STA);
271 WiFi.setMinSecurity(WIFI_AUTH_WEP);
272 Serial.println("Pripojovani_k_Wi-Fi...");
273 Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
274 Serial.println("Pripojeno_k_Wi-Fi!");
275
276 // Odeslani aktualnich nastaveni do Blynk
277 sendSettingsToBlynk();
278
279 // Prvni nacteni hodnot a inicializace last* promennych
280 readSensors();
281 lastTemp = temp;
282 lastHum = hum;
283 lastPress = press;
284 lastAirQuality = airQuality;
285

```

```

286 // Vynucene odeslani dat pri startu
287 sendSensorDataBlynk(true);
288 lastForcedSend = millis();
289
290 // Casovac pro kontrolu zmen kazdych 10 minut (pouze zmeny)
291 timer.setInterval(1000L, oneSecondTask);
292 timer.setInterval(900000L, []() { sendSensorDataBlynk(true); });
293 }
294
295
296
297
298
299 // -----
300 // Hlavni smycka programu
301 // -----
302 void loop() {
303     Blynk.run();
304     timer.run();
305 }

```

Listing 2: Pomocný soubor pro práci se soubory (settings_manager.cpp)

```

1 #include "settings_manager.h"
2 #include <LittleFS.h>
3 #include <ArduinoJson.h>
4
5 // Cesta k souboru s nastavenim
6 const char* SETTINGS_FILE = "/settings.json";
7
8 void loadSettings() {
9     Serial.println("Inicializuji LittleFS...");
10    if (!LittleFS.begin()) {
11        Serial.println("Chyba pri inicializaci LittleFS.");
12        return;
13    }
14    Serial.println("LittleFS inicializovano.");
15
16    if (!LittleFS.exists(SETTINGS_FILE)) {

```



```

17     Serial.println("Soubor settings.json nenalezen. Pouzivam
        vychozi nastaveni.");
18     saveSettings(); // Ulozi vychozi nastaveni do souboru
19     return;
20 }
21
22 File settingsFile = LittleFS.open(SETTINGS_FILE, "r");
23 if (!settingsFile) {
24     Serial.println("Chyba pri otevreni settings.json procteni
        .");
25     return;
26 }
27
28 StaticJsonDocument<512> doc; // Kapacita pro JSON dokument
29 DeserializationError error = deserializeJson(doc, settingsFile)
    ;
30 settingsFile.close();
31
32 if (error) {
33     Serial.print(F("Chyba pri parsovani settings.json:"));
34     Serial.println(error.f_str());
35     Serial.println("Pouzivam vychozi nastaveni.");
36     return;
37 }
38
39 tempThreshold = doc["tempThreshold"] | 30.0;
40 humThreshold = doc["humThreshold"] | 60.0;
41 pressThreshold = doc["pressThreshold"] | 1013.0;
42 airQualityThreshold = doc["airQualityThreshold"] | 150.0;
43 onOff = doc["onOff"] | false;
44
45 Serial.println("Nastaveni uspesne nacteno:");
46 Serial.printf("Teplota: %.2f C\n", tempThreshold);
47 Serial.printf("Vlhkost: %.2f %%\n", humThreshold);
48 Serial.printf("Tlak: %.2f hPa\n", pressThreshold);
49 Serial.printf("Kvalita vzduchu: %.2f ppm\n",
    airQualityThreshold);
50 Serial.printf("Regulace: %s\n", onOff ? "Zapnuta" : "Vypnuta"

```

```

    );
51 }
52
53 void saveSettings() {
54     if (!LittleFS.begin()) { // Zajisteni inicializace, pokud
55         nebyla drive
56         Serial.println("Chyba_pri_inicializaci_LittleFS_pro_
57             ukladani.");
58         return;
59     }
60
61     File settingsFile = LittleFS.open(SETTINGS_FILE, "w");
62     if (!settingsFile) {
63         Serial.println("Chyba_pri_otevirani_settings.json_pro_zapis
64             .");
65         return;
66     }
67
68     StaticJsonDocument<512> doc;
69     doc["tempThreshold"] = tempThreshold;
70     doc["humThreshold"] = humThreshold;
71     doc["pressThreshold"] = pressThreshold;
72     doc["airQualityThreshold"] = airQualityThreshold;
73     doc["onOff"] = onOff;
74
75     if (serializeJson(doc, settingsFile) == 0) {
76         Serial.println(F("Chyba_pri_zapisu_do_settings.json."));
77     } else {
78         Serial.println("Nastaveni_uspesne_ulozeno.");
79     }
80     settingsFile.close();
81 }

```